

Finding a longest nonnegative path in a constant degree tree

Sung Kwon Kim¹

Department of Computer Engineering, Chung-Ang University, 221 Huksuk-dong Dongjak-ku, Seoul 156-756, Republic of Korea

Received 16 July 2004; received in revised form 15 November 2004

Available online 4 January 2005

Communicated by F.Y.L. Chin

Abstract

A longest nonnegative path in an edge-weighted tree is a path such that the sum of edge weights on it is nonnegative and the number of edges on it is as large as possible. In this paper we show that if a tree has a constant degree, then its longest nonnegative path can be found in $O(n \log n)$ time, where n is the number of nodes. Previously known algorithms take $O(n \log^2 n)$ time.

© 2004 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Nonnegative paths; Trees

1. Introduction

Let $A = \langle a_1, \dots, a_n \rangle$ be an array of real numbers. For any $1 \leq i \leq j \leq n$, $\langle a_i, \dots, a_j \rangle$ is called a *subarray* of A . Its *length* is $j - i + 1$, its *sum* $a_i + \dots + a_j$, and its *average* $\frac{a_i + \dots + a_j}{j - i + 1}$. Given a threshold value θ , the problem of finding a longest subarray of A with its average at least θ has important applications in computational biology and bioinformatics, e.g., see [1,6].

The problem can be explained in another way as follows: As the average $\frac{a_i + \dots + a_j}{j - i + 1} \geq \theta$, this can be written as $(a_i - \theta) + \dots + (a_j - \theta) \geq 0$. From this, finding a longest subarray of A whose average is at least θ

now becomes finding a longest subarray of A' whose sum is nonnegative, where $A' = (a_1 - \theta, \dots, a_n - \theta)$. Finding a longest nonnegative subarray can be accomplished in $O(n)$ time [1,6].

A generalization of this in a tree is following: A tree $T = (V, E)$ consists of a set V of nodes and a set E of edges. Each edge $e \in E$ is associated with a weight, $w(e)$, which is a (positive, negative, or zero) real number. There exists a unique path P between two different nodes in a tree. The *length* of P is the number of edges in P , and its *weight*, denoted by $w(P)$, is the sum of the weights of edges in P . That is, $w(P) = \sum_{e \in P} w(e)$. P is called *nonnegative* if $w(P) \geq 0$. Finding a longest nonnegative path in a tree can be done in $O(n \log^2 n)$ time where $n = |V|$ by the algorithm in [7].

E-mail address: skkim@cau.ac.kr (S.K. Kim).

¹ Supported by the ITRI of Chung-Ang University.

The *degree* of a node is the number of edges incident on the node, and the *degree* of a tree is the maximum among the degrees of the nodes in it. The algorithm of [7] takes $O(n \log^2 n)$ time regardless of the degree of trees. We are interested in whether it is possible to reduce time complexity if a tree has a constant degree. We answer the question positively by giving an algorithm with running time $O(n \log n)$.

Our algorithm (with a slight modification) can be applied to any tree with a constant degree at least three; however, for ease of presentation, our attention will be restricted to trees of degree three.

Note 1. Wu et al. [7] developed an algorithm for the problem of finding in a tree a *length-constrained heaviest* path, i.e., a maximum-weight path whose length is at most some predefined threshold. In [7], each edge is associated with two values, *length* and *weight*, both of which are real numbers (negatives are allowed). The length (weight) of a path is the sum of its edge lengths (weights). Their algorithm is a divide-and-conquer algorithm: split a tree into (at most) three subtrees, recursively compute subsolutions, and combine them to obtain the solution. The *combine* step requires a sorting of $O(n)$ real numbers, which correspond to lengths of *paths*, taking $O(n \log n)$ time. Therefore, the time complexity of the whole algorithm is $O(n \log^2 n)$. If the sorting in the combine step can be done in linear time, the time complexity will reduce to $O(n \log n)$. They actually claimed that if the *edge* lengths are all integers in the range $[1..O(n)]$, integer sorting algorithms, e.g., counting sort [2], can be used and the time complexity will be $O(n \log n)$. *Even though the edge lengths are all $O(n)$, some paths may have lengths beyond $O(n)$.* So, the sorting cannot be accomplished in $O(n)$ time for some inputs.

Note 2. The algorithm in [7] finds a length-constrained heaviest path, but it can find a *weight-constrained longest* path by switching the roles of lengths and weights. A longest nonnegative path, which is the focus of our algorithm, is a weight-constrained longest path as we are trying to find a longest path whose weight ≥ 0 . In our algorithm each edge is of length one. The algorithm in [7] cannot find a longest nonnegative path in $O(n \log n)$ time, as pointed in Note 1, even if the input tree has a constant degree. Notice that since the roles of lengths and weights have been

switched, the sorting in the combine step is done with respect to path weights, not path lengths.

2. Structure of algorithm

Our algorithm is a recursive one based on a divide-and-conquer method. Before explaining the algorithm we need an important definition.

T is a tree of degree three. If a node v and its edges are removed from T , of degree three, then T is partitioned into (at most) three subtrees T_1, T_2 and T_3 , depending on the degree of v . Node v is called a *centroid* of T if $|T_i| \leq |T|/2$ for $i = 1, 2, 3$, where $|T|$ denotes the number of nodes in T . A tree has either one or two centroids; and if there are two, they must be adjacent [5]. If a tree has two centroids, one of them is chosen as *the* centroid.

The centroid of T can be found in $O(|T|)$ time [3, 4]. A well-known method starts with converting T into a (rooted) binary tree by choosing a node of degree one as the root. Let $T(v)$ for node v be the subtree consisting of v and all of its descendants. Compute $|T(v)|$ for every node v of T by traversing (the binary tree version of) T in postorder. If v is a leaf, then $|T(v)| = 1$; otherwise, $|T(v)| = 1 +$ the number of nodes in the left and right subtrees of v . During this procedure, the first node v that satisfies $|T(v)| \geq |T|/2$ is the centroid of T .

Our algorithm, as mentioned before, runs in a divide-and-conquer fashion:

Input: A tree T of degree three.

Output: The length of the longest nonnegative path in T (the path itself can be found by slightly modifying the algorithm).

[*Divide*] If T has only one node, then returns 0. Otherwise, find the centroid c of T , and remove c and its edges from T . At most three subtrees T_1, T_2 and T_3 are left. Depending on the degree of c , T_2, T_3 or both may be empty.

[*Conquer*] Recursively find the length, denoted by L_1 , of the longest nonnegative path in T_1 . In similar ways, recursively find L_2 and L_3 in T_2 and T_3 , respectively.

[*Combine*] Find the length, L_c , of the longest nonnegative path that is contained in T and passing

through c . Then, $\max\{L_1, L_2, L_3, L_c\}$ is the length of the longest nonnegative path in T .

Since L_i is the length of the longest nonnegative path that is entirely within T_i for $i = 1, 2, 3$, those paths that are considered in computing L_i do not contain c . L_c is the length of the longest nonnegative path among those that contain c . So, it is easy to see that $\max\{L_1, L_2, L_3, L_c\}$ is the length of the longest nonnegative path in T . Since L_1, L_2 , and L_3 are obtained recursively, we shall explain how to obtain L_c efficiently.

Let $W(n)$ be the worst-case time needed in computing the length of the longest nonnegative path in a tree with n nodes by our algorithm. Then, $W(1) = O(1)$, and for $n \geq 2$,

$$W(n) = W(n_1) + W(n_2) + W(n_3) + c_1 \cdot n + M(n), \quad (1)$$

where $n_i = |T_i|$ for $i = 1, 2, 3$. Note that $n = n_1 + n_2 + n_3 + 1$ and by the definition of centroid, $n_1, n_2, n_3 \leq n/2$.

In (1), $W(n_i)$ is the worst case time for recursively computing L_i for $i = 1, 2, 3$ by our algorithm, $c_1 \cdot n$ is the time for the *divide* step of locating the centroid of and partitioning the tree (c_1 is a constant), and $M(n)$ is the time for the *combine* step of computing L_c . If we show that $M(n) = O(n)$ in the remainder of this paper, then (1) becomes $W(n) = O(n \log n)$.

The paths that contain c are classified into those that contain c as one of their end nodes, and those that contain c as non-end nodes. Let L_c^c be the length of the longest nonnegative path among those paths that have c as their end nodes. To compute L_c^c , transform T into a rooted tree with root at c . While traversing the rooted tree in preorder starting at c , compute for each node the length and weight of the path from the root to it. If the length and weight of the path from the root to a node are known, those to each of its children can be obtained by adding one to its length and the weight of the edge to its weight. Having associated the lengths and weights to the nodes, we can find L_c^c in $O(n)$ time.

Finding the length of the longest nonnegative path among those that have c as non-end nodes is an important part of our algorithm, and it will be described in detail.

3. Finding the longest nonnegative path among those having c as non-end nodes

A path having c as a non-end node has their end nodes in two of T_1, T_2 , and T_3 . We shall explain how to obtain the length, $L_c^{1,2}$, of the longest nonnegative path among those paths that have one of its end nodes in T_1 and the other in T_2 . $L_c^{2,3}$ for the case of having end nodes in T_2 and T_3 , and $L_c^{1,3}$ for the case of having end nodes in T_1 and T_3 can be found in similar ways. Then, $L_c = \max\{L_c^c, L_c^{1,2}, L_c^{2,3}, L_c^{1,3}\}$.

For each node of T_1 , we compute the length and weight of the path between c and it. After collecting those that have length i , find the maximum of the weights among them and denote it by $A[i]$. That is, $A[i]$ for $1 \leq i \leq n_1$ is the maximum weight that can be obtained from T_1 by the length of i (i.e., by using i edges) from c . For some large i , there may be no path of length i and in this case $A[i] = -\infty$.

To compute the values of $A[1..n_1]$ efficiently, we first set $A[i] = -\infty$ for all $1 \leq i \leq n_1$. Visiting the nodes of T_1 in preorder, we compute l_v and w_v , the length and weight of the path from c to the current node v . If l_u and w_u for the parent u of v are known, then $l_v = l_u + 1$, $w_v = w_u + w((u, v))$. Whenever l_v and w_v are obtained, update $A[l_v] = \max\{A[l_v], w_v\}$. A can be computed $O(n_1)$ time. Similarly, $B[j]$ for $1 \leq j \leq n_2$ is defined to be the maximum weight that can be obtained from T_2 by the length of j from c . $B[1..n_2]$ can be computed in $O(n_2)$ time.

Lemma 1. $L_c^{1,2} = \max(\{i + j \mid A[i] + B[j] \geq 0\} \cup \{0\})$.

Lemma 1 is crucial for our algorithm and can be proved readily by the properties of A and B . Computing $L_c^{1,2}$ now becomes finding a pair i, j such that $i + j$ is largest while $A[i] + B[j] \geq 0$. If no such pair exists, $L_c^{1,2} = 0$. If all pairs of i and j are considered it takes $O(n_1 \cdot n_2)$. So, we need to develop a more efficient method.

For $j = 1, \dots, n_2$, define $i(j)$ as follows:

- $A[i(j)] + B[j] \geq 0$, and $A[i] + B[j] < 0$ for all $i(j) + 1 \leq i \leq n_1$; and
- If $A[i] + B[j] < 0$ for all $1 \leq i \leq n_1$, then $i(j) = -\infty$.

```

 $B[0] := \infty;$ 
 $j := k := n_2;$ 
output ( $j$ );
while ( $j > 0$ )
{
    while ( $B[j] \leq B[k]$ )  $j - -$ ;
    output ( $j$ );
     $k := j$ ;
}

```

Fig. 1. Computing $j_0, j_1, \dots, j_m$.

$i(j)$ is the largest index i such that, given j , $A[i] + B[j]$ is nonnegative. Then, Lemma 1 can be written as

$$L_c^{1,2} = \max(\{i(j) + j \mid 1 \leq j \leq n_2\} \cup \{0\}).$$

Now, the problem is how to compute $i(j)$ for all j efficiently. The following lemma is obvious from the definition of $i(j)$.

Lemma 2. For $1 \leq j, j' \leq n_2$, if $j < j'$ and $B[j] \leq B[j']$, then $i(j) + j < i(j') + j'$.

Index j' is said to *dominate* index j in Lemma 2, and the *dominated* indices are not to be considered in computing $L_c^{1,2}$.

To utilize Lemma 2, define $j_0, j_1, \dots, j_m$ as follows (m defined later):

- $j_0 = 0$;
- For $k = 1, 2, \dots, m$, j_k is the index such that $B[j_k] = \max\{B[j] \mid j_{k-1} + 1 \leq j \leq n_2\}$. If there are more than one such index, take the largest.
- Repeat as above until $j_k = n_2$, and let $m = k$.

Obviously, $j_m = n_2$. $B[j_1]$ is the maximum among $B[1], \dots, B[n_2]$, $B[j_2]$ is the maximum among $B[j_1 + 1], \dots, B[n_2]$, and $B[j_m]$ is the maximum among $B[j_{m-1} + 1], \dots, B[n_2]$. So, $B[j_1] > \dots > B[j_m]$.

Lemma 3. For $k = 1, 2, \dots, m$, j_k dominates all of $j_{k-1} + 1, \dots, j_k - 1$.

$j_0, j_1, \dots, j_m$ can be computed by the algorithm in Fig. 1.

Lemma 4. $L_c^{1,2} = \max(\{i(j_k) + j_k \mid 1 \leq k \leq m\} \cup \{0\})$.

```

1:  $i := n_1$ ;
2: while ( $i > 0$  and  $A[i] + B[j_1] < 0$ )  $i - -$ ;
3: if ( $i = 0$ ) {  $L_c^{1,2} := 0$ ; return; }
4:  $t := i + j_1$ ;
5:  $k := 1$ ;
6: while ( $k < m$ )
{
7:    $k++$ ;
8:   while ( $i > 0$  and  $A[i] + B[j_k] < 0$  and  $i + j_k > t$ )
      $i - -$ ;
9:   if ( $i = 0$ ) {  $L_c^{1,2} := t$ ; return; }
10:  if ( $A[i] + B[j_k] \geq 0$  and  $i + j_k > t$ )  $t := i + j_k$ ;
}
11:  $L_c^{1,2} := t$ ;
    return;

```

Fig. 2. Computing $L_c^{1,2}$.

By Lemma 4, $L_c^{1,2}$ can be determined by the algorithm in Fig. 2.

Statements 1–3 find $i(j_1)$. Decrement i , initially n_1 , by one until $A[i] + B[j_1] \geq 0$, and, at that time, we have $i(j_1) = i$. If no such i exists (i.e., if $A[i] + B[j_1] < 0$ for all i), $i = 0$ in statements 2 and 3, and thus, $i(j_1) = -\infty$. Hence, $L_c^{1,2} = 0$. As $B[j_1]$ is the maximum of $B[1], \dots, B[n_2]$, if $A[i] + B[j_1] < 0$ for all i , then $A[i] + B[j] < 0$ for all i, j . We do not need to consider $j_2, \dots, j_m$.

In statement 4, $i(j_1) + j_1$ is stored into t , which will be the value of $L_c^{1,2}$ at the conclusion of the algorithm.

The while of statements 6–10 updates t , considering $B[j_2], \dots, B[j_m]$ in this order. Statement 8 tries to determine if there exists index i , with j_k fixed, such that $A[i] + B[j_k]$ is nonnegative and $i + j_k$ is larger than the current value of t . In the while loop of statement 8, decrement i if all three conditions are satisfied. If the while loop terminates with $A[i] + B[j_k] \geq 0$, then $i(j_k) = i$. So, t is updated in statement 10. If $i + j_k = t$, the while loop also terminates. In this case, however, we do not need to compute $i(j_k)$ as t will not increase because $i(j_k) + j_k \leq t$. If $i = 0$ and the while loop ends, then the algorithm stops and the current t becomes $L_c^{1,2}$. $j_{k+1}, \dots, j_m$ are not to be considered by a similar reasoning for the statements 1–3 above.

If $k = m$ in statement 6, go to statement 11 and let $L_c^{1,2}$ be the current t .

Note that in the algorithm one need not ever increase i . This is because for $2 \leq k \leq m$, $A[i] + B[j_{k-1}] < 0$ implies $A[i] + B[j_k] < 0$.

Lemma 5. $L_c^{1,2} = \max(\{i + j \mid A[i] + B[j] \geq 0\} \cup \{0\})$ can be computed in $O(n_1 + n_2)$ time.

Proof. Correctness has been proved. $A[\cdot]$ and $B[\cdot]$ can be computed in $O(n_1)$ and $O(n_2)$ time, respectively. The algorithm in Fig. 1 can be done $O(n_2)$ time. The algorithm in Fig. 2 can be done $O(n_1 + m)$ time, which is $O(n_1 + n_2)$ as $m \leq n_2$. $\square$

$L_c^{2,3}$ and $L_c^{1,3}$ can be computed as $L_c^{1,2}$ is computed in Lemma 5. From these and L_c^c , L_c is obtained.

Lemma 6. L_c can be computed $O(n)$ time.

Lemma 6 shows that $M(n)$ in Eq. (1) is $O(n)$. Therefore, $W(n) = O(n \log n)$.

This completes the result of this paper.

Theorem 1. For a tree of degree three with n nodes, its longest nonnegative path can be found $O(n \log n)$ time.

References

- [1] L. Allison, Longest biased interval and longest nonnegative sum interval, *Bioinformatics* 9 (10) (2003) 1294–1295.
- [2] T.H. Cormen, C.E. Leiserson, R.L. Rivest, *Introduction to Algorithms*, MIT Press, Cambridge, MA, 1994.
- [3] A.J. Goldman, Optimal center location in simple networks, *Transportation Science* 5 (1971) 212–221.
- [4] O. Kariv, S.L. Hakimi, An algorithmic approach to network location problem. I. The p-centers, *SIAM J. Appl. Math.* 37 (1979) 513–538.
- [5] D.E. Knuth, *Fundamental Algorithms*, second ed., The Art of Programming, vol. 1, Addison-Wesley, Reading, MA, 1973.
- [6] L. Wang, Y. Xu, SEGID: Identifying interesting segments in (multiple) sequence alignments, *Bioinformatics* 19 (2) (2003) 297–298.
- [7] B.Y. Wu, K.-M. Chao, C.Y. Tang, An efficient algorithm for the length-constrained heaviest path problem on a tree, *Inform. Process. Lett.* 69 (1999) 63–67.